

实验单元3： CSRF

3.1 实训说明

3.1.1 实训目的

掌握利用 burpsuite 进行 CSRF 渗透的步骤。

3.1.2 背景知识

- 1.掌握 CSRF 的基本原理
- 2.掌握 CSRF 漏洞的判断
- 3.掌握 CSRF 的基本防御

3.1.3 实习时长

2 个学时

3.2 实训环境

3.2.1 实训器材

kali linux 2024

3.2.2 网络结构



kali linux 2024 虚拟机（）

本实验中，请实验老师根据学校组网与实验环境分配 IP。

3.3 实训内容

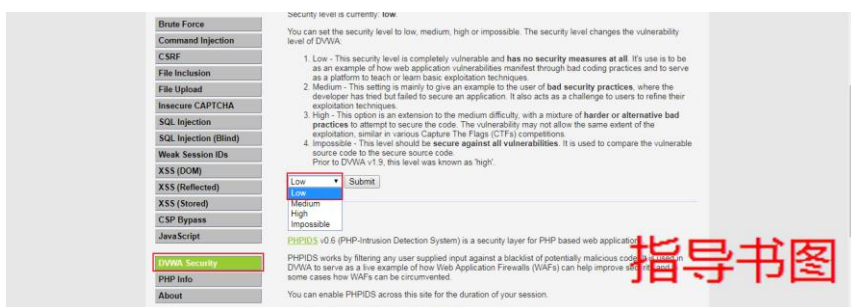
3.3.1 实训准备

打开 Apache2 和 mariadb 服务。

登录 DVWA, 在浏览器输入 <http://127.0.0.1/DVWA/login.php>, 转到 DVWA 的登录页面, 使用用户名 admin, 密码 password 进行登录。

3.3.2 CSRF-Low

1. DVWA 1.9 的代码分为四种安全级别: Low, Medium, High, Impossible。选择 Low 安全级别。



2. 服务器收到修改密码的请求后, 会检查参数 password_new 与 password_conf 是否相同, 如果相同, 就会修改密码, 并没有任何的防 CSRF 机制。

```
<?php
if( isset( $_GET[ 'Change' ] ) ) {
    // Get input
    $pass_new = $_GET[ 'password_new' ];
    $pass_conf = $_GET[ 'password_conf' ];

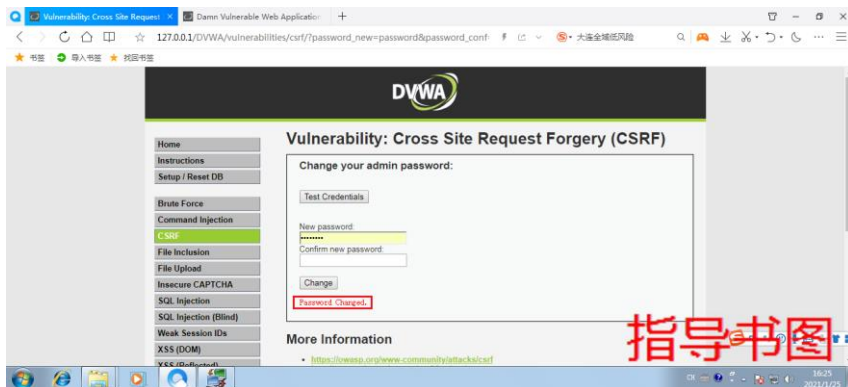
    // Do the passwords match?
    if( $pass_new == $pass_conf ) {
        // They do!
        $pass_new = (isset($GLOBALS['__mysqli_stor']) && is_object($GLOBALS['__mysqli_stor'])) ? mysqli_real_escape_string($GLOBALS['__mysqli_stor'], $pass_new) : (isset($GLOBALS['__mysqli_stor']) ? addslashes($pass_new) : $pass_new);

        // Update the database
        $insert = "UPDATE users SET password = '$pass_new' WHERE user = '" . $_SESSION['username'] . "'";
        $result = mysqli_query($GLOBALS['__mysqli_stor'], $insert) or die( 'oops' );
        // Feedback for the user
        echo "Password Changed./pre>";
    }
    else {
        // Issue with passwords matching
        echo "Passwords did not match./pre>";
    }
}

((is_null($_mysqli_res) || $_mysqli_res === false) ? false : $_mysqli_res);
```

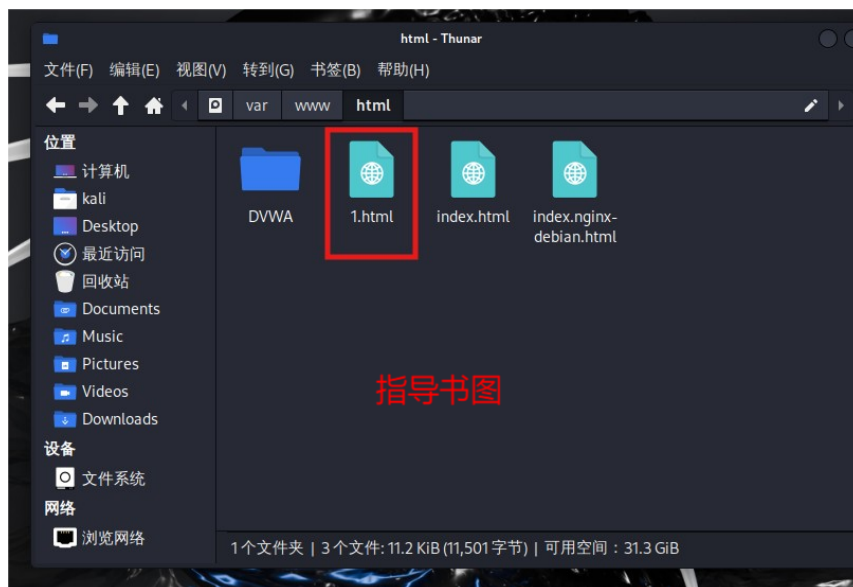
3. 构造链接

http://127.0.0.1/DVWA/vulnerabilities/csrf/?password_new=password&password_conf=password&Change=Change# 当受害者点击了这个链接, 密码就会被改成 password。

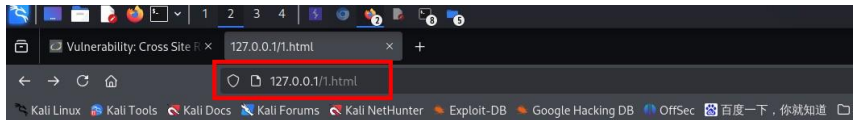


4. 现实攻击场景下，需要事先在公网上传一个攻击页面（也可以伪装成一张图片），并进行网址处理，诱骗受害者去访问，真正能够在受害者不知情的情况下完成 CSRF 攻击。例如以下的 1.html，下面是具体代码。

```
  
  
<h1>404</h1>  
  
<h2>file not found.</h2>
```

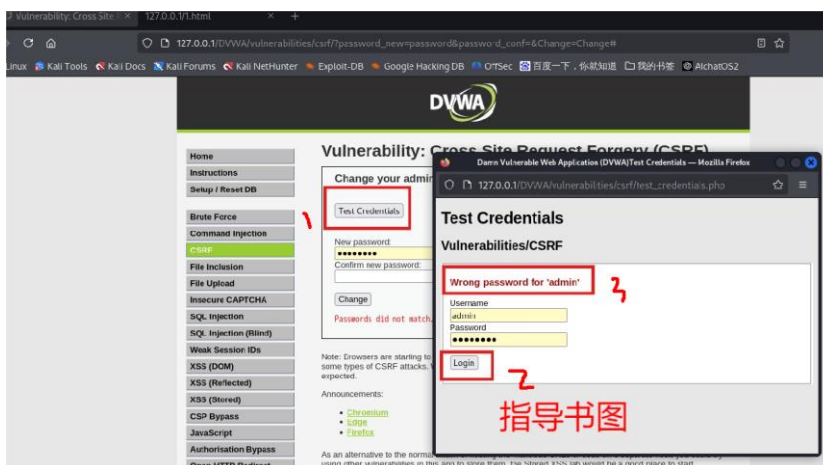


5. 当受害者访问 1.html 时，会误认为是自己点击的是一个失效的 url，但实际上已经遭受了 CSRF 攻击，密码已经被修改为了 123456。



404

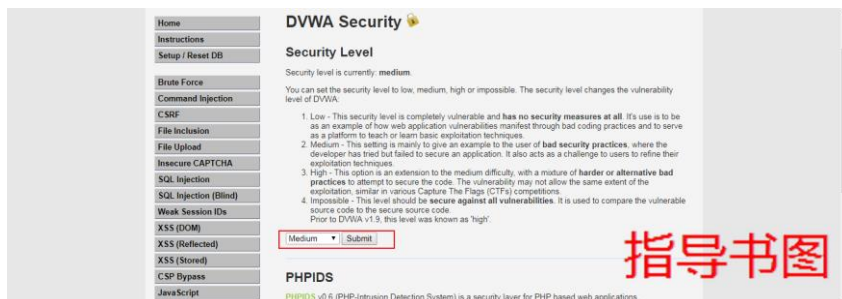
file not found.



注意：这一步实验完应将 admin 的密码还原为 password

3.3.3 CSRF-Medium

1. DVWA 1.9 的代码分为四种安全级别: Low, Medium, High, Impossible。选择 Medium 安全级别。

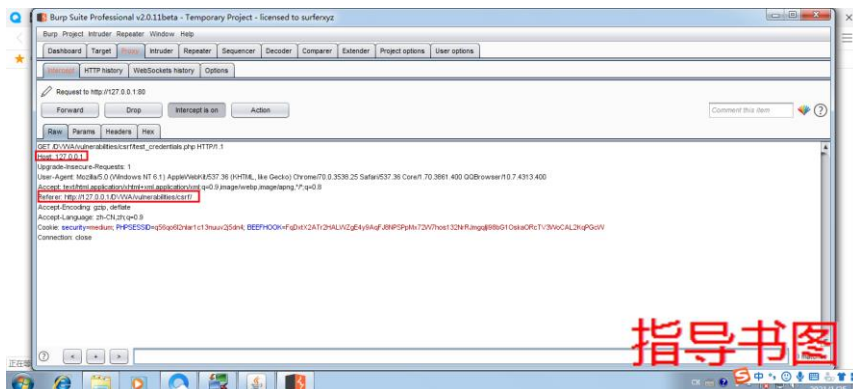


2. Medium 级别的代码检查了保留变量 HTTP_REFERER (http 包头的 Referer 参数的值, 表示来源地址) 中是否包含 SERVER_NAME (http 包头的 Host 参数, 及要访问的主机名, 希望通过这种机制抵御 CSRF 攻击。

```
<?php
if( isset( $_GET[ 'Change' ] ) ) {
    // Checks to see where the request came from
    if( stripslashes( $_SERVER[ 'HTTP_REFERER' ] ) != $_SERVER[ 'SERVER_NAME' ] ) {
        // Get input
        $pass_new = $_GET[ 'password_new' ];
        $pass_conf = $_GET[ 'password_conf' ];
        // Do the password match?
        if( $pass_new == $pass_conf ) {
            // They do!
            $pass_new = (isset($_GLOBALS["__mysqli_ston"]) && is_object($_GLOBALS["__mysqli_ston"])) ? mysqli_real_escape_string($_GLOBALS["__mysqli_ston"],
            $pass_new ) : (trigger_error("MySQLConverteeToo! File the mysqli_real_escape_string() call! This code does not work.", E_USER_ERROR)) ? "" : "");
            $pass_new = md5( $pass_new );
            // Update the database
            $insert = "UPDATE users SET password = '$pass_new' WHERE user = '" . driveCurrentUser() . "'";
            $result = mysqli_query($_GLOBALS["__mysqli_ston"], $insert ) or die( '
```

int eregi(string pattern, string string): 检查 string 中是否含有 pattern (不区分大小写), 如果有返回 True, 反之 False。

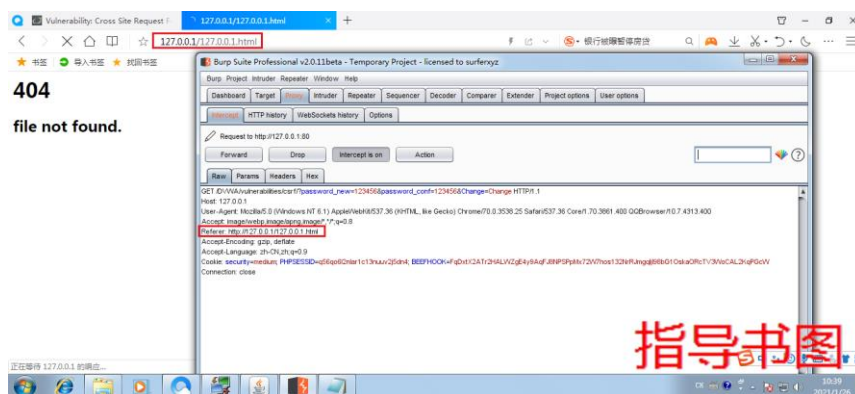
3. Medium 级别的代码检查了保留变量 HTTP_REFERER (http 包头的 Referer 参数的值, 表示来源地址) 中是否包含 SERVER_NAME (http 包头的 Host 参数, 及要访问的主机名), 希望通过这种机制抵御 CSRF 攻击。



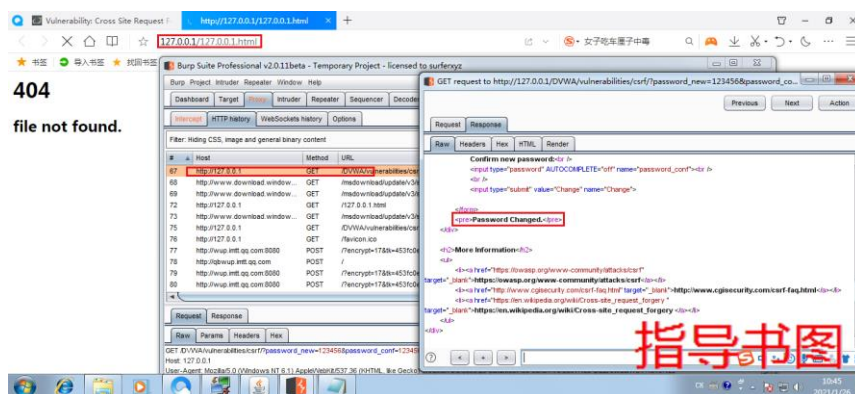
4. 过滤规则是 http 包头的 Referer 参数的值中必须包含主机名（这里是 127.0.0.1）我们可以将攻击页面命名为 127.0.0.1.html。

```
(kali@kali)-[/var/www/html]
$ sudo cp 1.html 127.0.0.1.html
(kali@kali)-[/var/www/html]
$ ll
总计 36
-rw-r--r-- 1 kali kali 128 3月27日 14:11 123.html
-rw-r--r-- 1 root root 182 3月27日 14:23 127.0.0.1.html
-rw-r--r-- 1 kali kali 182 3月27日 14:18 1.html
-rw-r--r-- 1 root root 129 3月27日 14:09 2.html
drwxr-xr-x 12 root root 4096 3月27日 12:11 DVWA
-rw-r--r-- 1 root root 10703 11月30日 07:33 index.html
-rw-r--r-- 1 root root 615 11月30日 07:29 index.nginx-debian.html
```

5. 通过 burpsuite 抓包，可以看到 Referer 参数完美绕过过滤规则。



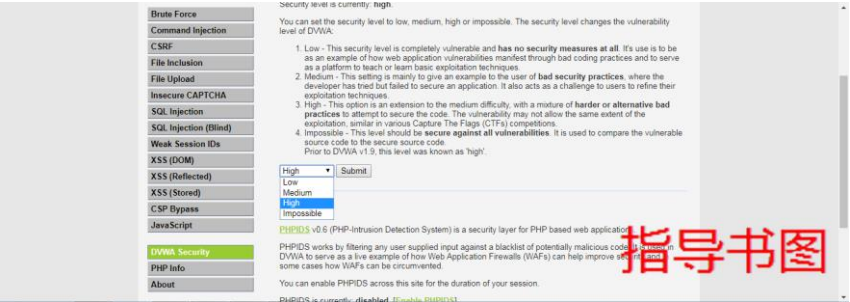
6. 密码修改成功



注意：这一步实验完应将 admin 的密码还原为 password

3.3.4 CSRF-High

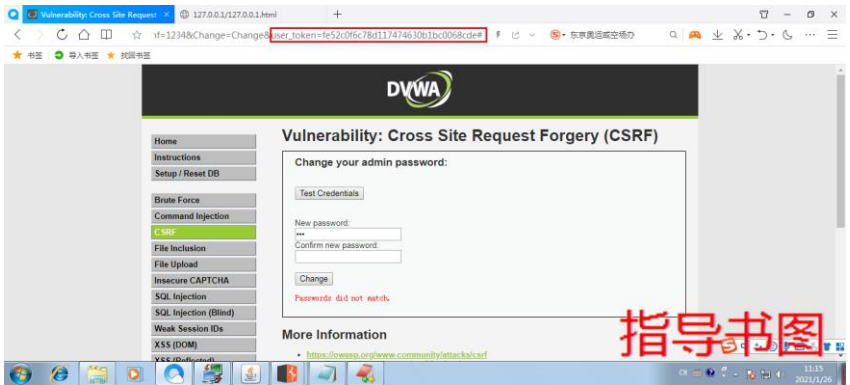
1. DVWA 1.9 的代码分为四种安全级别：Low, Medium, High, Impossible。选择 High 安全级别。



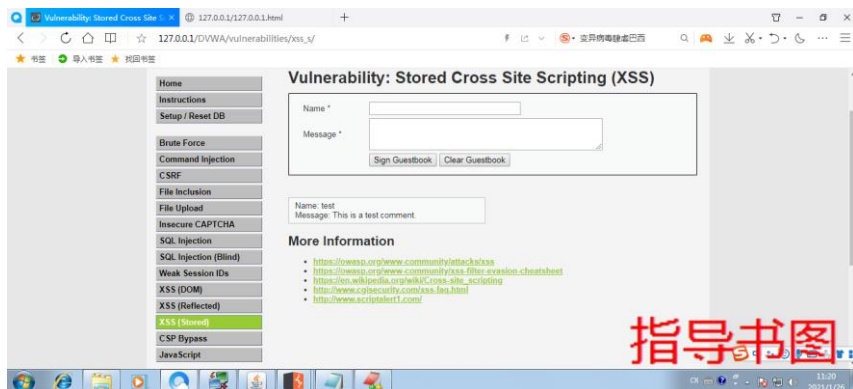
2. 可以看到，High 级别的代码加入了 Anti-CSRF token 机制，用户每次访问改密页面时，服务器会返回一个随机的 token，向服务器发起请求时，需要提交 token 参数，而服务器在收到请求时，会优先检查 token，只有 token 正确，才会处理客户端的请求。

```
<?php
if(isset($_GET['Change'])) {
    // Check Anti-CSRF token
    checkToken($_REQUEST['user_token'], $_SESSION['session_token'], 'index.php');
    // Get input
    $pass_new = $_GET['password_new'];
    $pass_conf = $_GET['password_conf'];
    // Do the passwords match?
    if($pass_new == $pass_conf) {
        // They do!
        $pass_new = ((isset($_GLOBALS["__mysqli_ston"]) && is_object($_GLOBALS["__mysqli_ston"]))) ? mysqli_real_escape_string($_GLOBALS["__mysqli_ston"], $pass_new) : ((trigger_error("MySQLConverterToo: Fix the mysqli_escape_string() call! This code does not work.", E_USER_ERROR)) > "" ? "" : "");
        $pass_new = md5($pass_new);
        // Update the database
        $insert = "UPDATE users SET password = '$pass_new' WHERE user = '" . $_SESSION['user'] . "'";
        $result = mysqli_query($_GLOBALS["__mysqli_ston"], $insert) or die(' <pre>'. ((is_object($_GLOBALS["__mysqli_ston"]))) ? mysqli_error($_GLOBALS["__mysqli_ston"]) : ((trigger_error("MySQLConverterToo: Fix the mysqli_error() call! This code does not work.", E_USER_ERROR)) > "" ? "" : ""));
        // Feedback for the user
        echo "<pre>Password Changed.</pre>";
    }
    else {
        // Issue with passwords matching
        echo "<pre>Passwords did not match.</pre>";
    }
}
// Generate Anti-CSRF token
generateSessionToken();
```

3. 因为该请求是 get 请求，所以 token 验证会被放在请求 URL 中，随便输入密码验证一下，可以看到，在请求的 URL 中最末尾加入了 token。

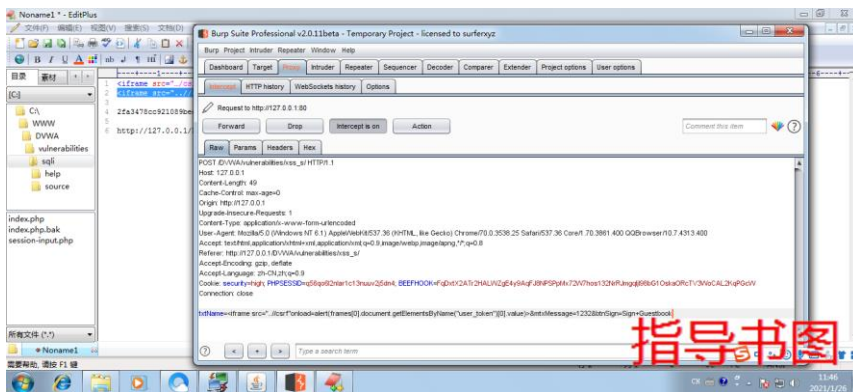


- 试着去构造一个攻击页面，将其放置在攻击者的服务器，引诱受害者访问，从而获得 token 值，并向服务器发送改密请求，完成攻击。但是，浏览器并不允许跨域请求，因此，需要利用 xss 漏洞，点击 XSS(Stored)，我们需要构造一条语句来获取 token。

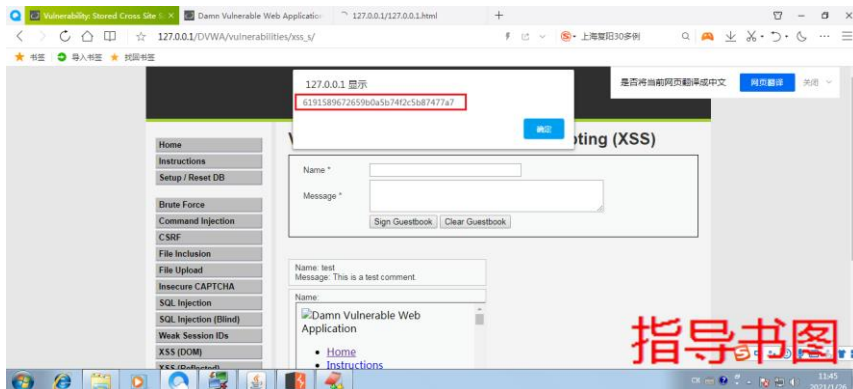


- 由于有字符数限制，可以利用 burp suite 进行抓包，然后将获取 cookie 的 js 代码赋值给 name 参数 `<iframe src='../../csrf' onload=alert(frames[0].document.getElementsByName('user_token')[0].value)>`

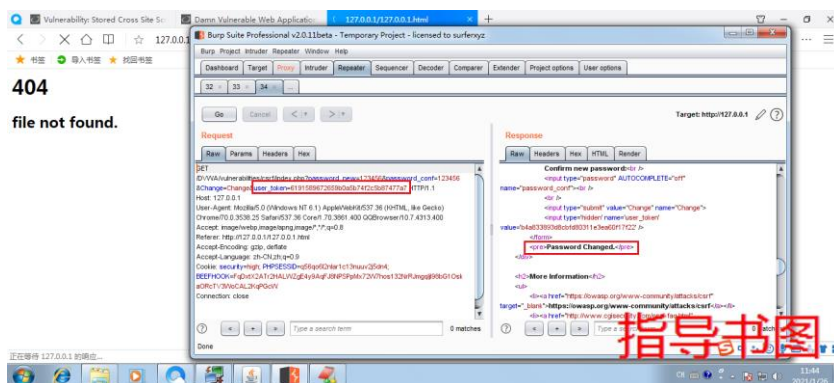
批注 [y1]: 注意参数里的空格

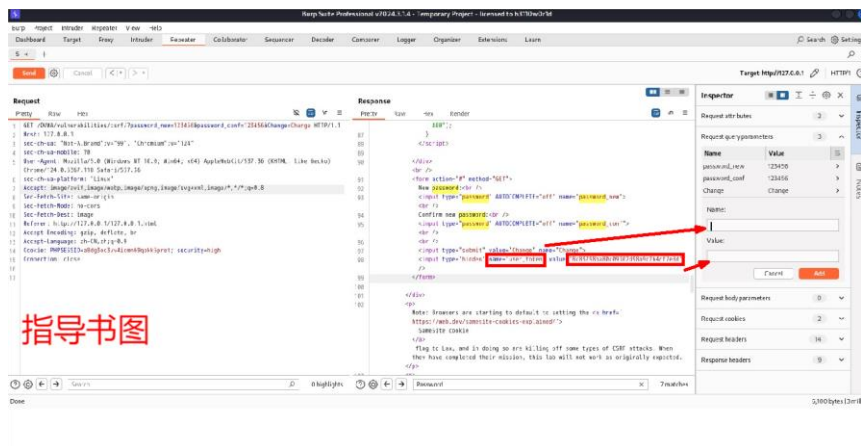


- Name 存在 XSS 漏洞，于是抓包，改参数，成功弹出 token。



7. 获取到 token 值后，打开 Burp Suite 工具抓取数据包，在浏览器中访问 127.0.0.1/127.0.0.1.html，之后打开 Burp Suite 找到刚刚访问网址 127.0.0.1/127.0.0.1.html 抓取到的数据包，并发送到 Repeater，将 user_token 的值进行修改，并点击“Go”，返回的结果显示密码修改成功。（如果失败，刷新 xss 的 token 值，重新更新 user_token 的值进行发送）





3.4 总结与思考

3.4.1 实训总结

这部分内容是关于 CSRF 漏洞的操作过程，主要包括：

- GET 方式 CSRF 漏洞的利用方式。
- POST 方式 CSRF 漏洞的利用方式。
- 利用 burpsuit 生成 CSRF 利用代码。

3.4.2 思考题

问题 1：简要描述 CSRF 与 XSS 的区别？

问题 2：出现 CSRF 漏洞的常见场景？

